

DragonLab Syllabus

Charles Averill

Spring 2025

Contents

1 Learning Objectives	1
2 Reading List	1
2.1 Volume 1: Logical Foundations	1
2.2 Volume 2: Programming Language Foundations	2
2.3 Volume 5: Verifiable C	2
3 Schedule	2

This document outlines the structure of the course, including material covered and schedule.

1 Learning Objectives

Students should obtain working knowledge of the following upon completion of all units:

1. Basic syntax, data types, patterns, and proof strategies for the Rocq proof system
2. The Curry-Howard Isomorphism as it relates to program verification
3. Basic reasoning strategies for imperative programs, primarily
 - Large- and small-step operation semantics
 - Hoare logic and Separation logic
4. The simply-typed lambda calculus and its derivatives
5. The basics of C program verification using the Verified Software Toolchain

Some stretch goals:

- Functional correctness models (Volume 3: Verified Functional Algorithms)

2 Reading List

The primary source material for this course is [Software Foundations](#). The following ordered list of chapters illustrates the flow of the course.

2.1 Volume 1: Logical Foundations

Chapters 1-7: - Functional Programming - Proof by Induction - Lists - Polymorphism - Basic proof tactics - Logic - Inductive Propositions

Chapter 9: Proof Objects

Chapter 12: Simple Imperative Programs

2.2 Volume 2: Programming Language Foundations

Chapters 2-3: Hoare Logic

Chapter 5: Small-step Operational Semantics

Chapters 6-8: - Type Systems - The Simply-Typed Lambda Calculus - Properties of the STLC

2.3 Volume 5: Verifiable C

Chapters 1-2: Introduction and Linked Lists

3 Schedule

The above material is to be covered in a long semester (roughly 12 weeks). Much of the material will only be covered qualitatively, with the instructor stepping through at least one entire chapter for each 90-minute lecture.

Week	Chapters	Recommended extra reading
1	1.1, 1.2	Tour of OCaml
2	1.3, 1.4	
3	1.5, 1.6	Coq Tactics in Plain English
4	1.7	
5	1.9	The Curry-Howard Isomorphism
6	1.12, 2.2	An Axiomatic Basis for Computer Programming
7	2.3	
8	2.5	
9	2.6, 2.7	
10	2.8	
11	5.1	
12	5.2	